

Features implemented in the project from the presentation

1. **Basic C++ Syntax, Classes, Object-Oriented Design, Encapsulation, and Constructors** *(Presentations 1–2)*

Implemented multiple custom classes such as `Player`, `Unit`, `AI`, `Utils`, and configuration systems.

Applied encapsulation by storing data inside private members and exposing controlled access through public methods.

Used constructors to initialize objects and game entities with predefined states.

Designed the project using object-oriented principles including abstraction, modularity, and separation of responsibilities.

Implemented enums (`UnitType`) for structured and type-safe game logic.

2. **Standard Template Library (STL), Sequence Containers, Iterators Ranges and Algorithms** *(Presentation 3)*

`std::vector<Unit>` used for squads in `Player`.

Modern ranges and algorithms for checking if any unit is alive, counting them, `std::views::iota` for iterating indices without additional loops

`std::views::reverse` to draw the map from top Y to bottom Y.

Dynamic size, automatic memory management, and easy iteration.

Utilized iterators and range-based loops for traversing and manipulating containers efficiently.

`std::clamp` used to keep coordinates inside

3. **Random Number Generation `<random>`, numeric limits, optional, and Modern Random Engines** *(Presentation 4)*

`thread_local std::mt19937` with `std::random_device` for per-thread PRNG.

`std::uniform_int_distribution<int>` for damage between `minAttack` and `maxAttack` in `Unit::Attack`.

Implemented modern random number generation using `<random>` instead of outdated `rand()` approaches where appropriate.

Generated randomized spawn positions, attack variability.

Used modern <random> APIs with `std::mt19937` and `std::uniform_int_distribution` for combat damage, while classic `rand` is used for initial spawn randomness.

4. **Threads and Dynamic Memory Allocation** *(Presentation 5)*

`main()` uses `system("start ... uv run uvicorn main:app --reload")` to start the FastAPI/uvicorn server.

Then uses `std::this_thread::sleep_for(std::chrono::milliseconds(3500))`; to wait for the server to be ready instead of `Sleep`.

Used <threads> and <chrono> instead of `Sleep`.

Utilized STL containers (`std::vector`) which internally rely on dynamic memory allocation and automatic resource management.

Applied modern C++ memory-safe design principles by avoiding manual raw-memory manipulation wherever possible.

Reduced risks of memory leaks through container-managed object lifetimes.

5. **Pass-by-Value, Pass-by-Reference, Refactoring, Modularization, and Lambda Functions** *(Presentation 6)*

Used pass-by-reference to improve performance and avoid unnecessary object copying.

Used pass-by-value where object independence and immutability were desired.

Used lambdas in most functions including the reading json file.

Separated responsibilities into dedicated classes - refactoring (`AI`, `Player`, `Utils`, `Unit`, `Config`).

6. Some additional advanced features *(Presentation 8)*

`std::ranges::any_of`,

`std::ranges::count_if`

`std::views::iota` and `std::views::reverse`

`std::optional`

